
Model Fusion via Optimal Transport

Sidak Pal Singh¹ Martin Jaggi¹

Abstract

Combining different models is a widely used paradigm in machine learning applications. While the most common approach is to form an ensemble of models and average their individual predictions, this approach is often rendered infeasible by given resource constraints in terms of memory and computation, which grow linearly with the number of models.

We present a layer-wise model fusion algorithm for neural networks that utilizes optimal transport to (soft-) align neurons across the models before averaging their associated parameters. We discuss two main strategies for fusing neural networks in this “one-shot” manner, without requiring any retraining. Next, we illustrate how this significantly outperforms vanilla averaging on convolutional networks (like VGG11), residual networks (like RESNET18), and multi-layer perceptrons, on CIFAR10 and MNIST. Finally, we show applications to transfer tasks (where our fused model even surpasses the performance of *both* the original models) as well as for compressing models.

Code will be made available under the following link <https://github.com/modelfusion>.

1. Introduction

If two neural networks had a child, what would be its weights? In this work, we study the fusion of two *parent* neural networks—which were trained differently but have the same number of layers—into a single *child* network. We further focus on performing this operation in a one-shot manner, based on the network weights only, so as to minimize the need of any retraining.

This fundamental operation of merging two neural networks into one contrasts other widely used techniques for combining machine learning models:

¹EPFL. Correspondence to: Sidak Pal Singh <sidak.singh@epfl.ch>, Martin Jaggi <martin.jaggi@epfl.ch>.

Ensemble methods have a very long history in the literature, as well as in many successful machine learning deployments. They combine the outputs of several different models as a way to improve the prediction performance and robustness. However, this requires maintaining the K trained models and running each of them at test time (say, in order to average their outputs). This approach thus quickly becomes infeasible for many applications with limited computational resources, especially in view of the ever-growing size of modern deep learning models.

The simplest way to fuse two parent networks into a single network of the same size is direct *weight averaging*, which we refer to as vanilla averaging; here for simplicity, we assume that all network architectures are identical. Unfortunately, neural networks are typically highly redundant in their parameterizations, so that there is no one-to-one correspondence between the weights of two different neural networks, even if the networks would describe the very same function of the input. In practice, vanilla weight averaging is known to perform very poorly on trained neural networks whose weights differ non-trivially.

Finally, a third way to combine two models is *distillation*, where one network is retrained on its training data, by using along with, the output predictions of the other network on those samples. Such a scenario is considered infeasible in our setting, as we aim for approaches not requiring the sharing of training data. This requirement is particularly crucial if the training data is to be kept private, like in federated learning applications, or is unavailable due to e.g. legal reasons.

Contributions. We propose a novel layer-wise approach of aligning the neurons and weights of two differently trained models, for fusing them into a single model of the same architecture. Our method relies on optimal transport (OT) (Kantorovich, 1942; Monge, 1781), to minimize the transportation cost of neurons present in the layers of individual models, measured by the similarity of activations or incoming weights. The resulting layer-wise averaging scheme can be interpreted as computing the Wasserstein barycenter (Agueh & Carlier, 2011; Cuturi & Doucet, 2014) of the probability measures defined at the corresponding layers of the parent models.

We empirically demonstrate that our method succeeds in the

one-shot merging of networks of different weights, and in all scenarios significantly outperforms vanilla averaging.

More surprisingly, we also show that our method succeeds in merging two networks that were trained for slightly different tasks (such as using a different set of labels). The method is able to “inherit” new abilities unique to one of the parent networks, while outperforming the same parent network on the task associated with the other network.

Our experiments cover standard image classification datasets, MNIST & CIFAR10, for neural networks such as multi-layer perceptrons (MLPs), convolutional neural networks (CNNs): VGG11 (Simonyan & Zisserman, 2014), and residual networks: RESNET18 (He et al., 2016).

Extensions and Applications. Our techniques are straightforward to extend to the case of merging more than two parent networks. Also, while we require the overall number of layers to be identical, we do allow corresponding layers between the several networks to be of different sizes.

The method serves as a new building block for enabling several use-cases. For instance, (1) The adaptation of a global model to personal training data. (2) Fusing the parameters of a bigger model into a model of smaller size. (3) Federated or decentralized learning applications, where training data is not allowed to be shared due to privacy reasons or simply due to its large size. In general, improved model fusion techniques such as ours have strong potential towards encouraging model exchange as opposed to data exchange, to improve privacy & reduce communication costs.

2. Related Work

Ensembling. Ensemble methods (Breiman, 1996; Schapire, 1999; Wolpert, 1992) have long been in use in deep learning and machine learning in general. However, given our goal is to obtain a single model, it is assumed infeasible to maintain and run several trained models as needed here.

Distillation. Another line of work by Buciluă et al. (2006); Hinton et al. (2015); Schmidhuber (1992) proposes distillation techniques. Here the key idea is to employ the knowledge of a pre-trained teacher network (typically larger and expensive to train) and transfer its abilities to a smaller model called the student network. During this transfer process, the goal is to use the relative probabilities of misclassification of the teacher as a more informative training signal.

While distillation also results in a single model, the main drawback is its computational complexity—the distillation process is essentially as expensive as training the student network from scratch, and also involves its own set of hyperparameter tuning. In addition, distillation still requires sharing the training data with the teacher, which we avoid here.

In a different line of work, Shen et al. (2018) propose an

approach where the student network is forced to produce outputs mimicking the teacher networks, by utilizing Generative Adversarial Network (Goodfellow et al., 2014). This still does not resolve the problem of high computational costs involved in this kind of knowledge transfer. Further, it does not provide a principled way to aggregate the parameters of different models, which we provide via our method.

Relation to other network fusion methods. Several studies have investigated a method to merge two trained networks into a single network without the need for retraining (Leontev et al., 2018; Smith & Gashler, 2017; Utans, 1996). Leontev et al. (2018) proposed using Elastic Weight Consolidation, which formulates an assignment problem on top of the diagonal approximations to the Hessian matrices of each of the two parent neural networks. Their proposed method however only works when the weights of the parent models are already close, i.e. share a significant part of the training history (Smith & Gashler, 2017; Utans, 1996), by relying on SGD with periodic averaging, also known as local SGD (Stich, 2019). The empirical results of Leontev et al. (2018) nevertheless do not improve over vanilla averaging.

Alignment-based methods. Recently, Yurochkin et al. (2019) independently proposed a Bayesian non-parametric framework that considers matching the neurons of different MLPs in federated learning. In a concurrent work, Wang et al. (2020) extend (Yurochkin et al., 2019) to more realistic networks including CNNs, for the specific focus of federated learning. In contrast, we develop our method from the lens of optimal transport (OT), which lends us a simpler approach by utilizing Wasserstein barycenters. The method of aligning neurons employed in both lines of work form instances of a particular choice of ground metric in OT. Overall, we consider model fusion in general, beyond federated learning. For e.g., we show applications of fusion under size constraints (i.e., model compression) as well as the compatibility of our method to serve as an initialization for distillation. From a practical point, our approach applies to ResNets, which are not currently handled in the above methods.

Wasserstein barycenters. OT has recently received a lot of interest in machine learning, since the entropic regularization introduced by Cuturi (2013), which has made it more viable for practical use. In the past, Wasserstein barycenters have been used in problems concerning clustering (Cuturi & Doucet, 2014), graphics (Solomon et al., 2015), sentence representations (Singh et al., 2018), Bayesian averaging (Backhoff-Veraguas et al., 2018), averaging predictions for multi-label settings (Dognin et al., 2019), etc. The application of Wasserstein barycenters for averaging the weights of neural networks has—to our knowledge—not been considered in the past.

3. Background on Optimal Transport (OT)

We present a short background on OT in the discrete case, and in this process set up the notation for the rest of the paper. OT gives a way to compare two probability distributions defined over a ground space $\mathcal{S}$, provided an underlying distance or more generally the cost of transporting one point to another in the ground space. Next, we describe the linear program (LP) which lies at the heart of OT.

LP Formulation. First, let us consider two empirical probability measures μ_A and μ_B denoted by a weighted sum of Diracs, i.e., $\mu_A = \sum_{i=1}^{n_A} \alpha_i \delta(s_A^{(i)})$ and $\mu_B = \sum_{j=1}^{n_B} \beta_j \delta(s_B^{(j)})$. Here $\delta(s)$ denotes the Dirac (unit mass) distribution at point $s \in \mathcal{S}$ and the set of points $\mathcal{S} = (s^{(1)}, \dots, s^{(n)}) \in \mathcal{S}^n$. The weight $\alpha = (\alpha_1, \dots, \alpha_{n_A})$ lives in the probability simplex $\Sigma_{n_A} := \{p \in \mathbb{R}_+^{n_A} \mid \sum_{i=1}^{n_A} p_i = 1\}$ (and similarly β). Further, let C_{ij} denote the ground cost of moving point $s^{(i)}$ to $s^{(j)}$. Then the optimal transport between μ_A and μ_B can be formulated as solving the following linear program.

$$\text{OT}(\mu_A, \mu_B; C) := \min_{T \in \mathbb{R}_+^{(n_A \times n_B)}} \langle T, C \rangle \quad (1)$$

s.t. $T \mathbf{1}_{n_B} = \alpha, T^T \mathbf{1}_{n_A} = \beta$

Where, $\langle T, C \rangle := \text{tr}(T^T C) = \sum_{ij} T_{ij} C_{ij}$ is the Frobenius inner product of matrices. The optimal $T \in \mathbb{R}_+^{(n_A \times n_B)}$ is called as the *transportation matrix* or *transport map*, and T_{ij} represents the optimal amount of mass to be moved from point $s^{(i)}$ to point $s^{(j)}$. A real-world example where this holds is the problem of transporting a given amount of goods, like bread, from certain bakeries to meet the demands at some cafes, so that the overall cost is minimal.

Wasserstein Distance. In the case where $\mathcal{S} = \mathbb{R}^d$ and the cost is defined with respect to a metric D_S over $\mathcal{S}$ (i.e., $C_{ij} = D_S(s^{(i)}, s^{(j)})^p$ for any i, j), OT establishes a distance between probability distributions. This is called the p -Wasserstein distance and is defined as $\mathcal{W}_p(\mu_A, \mu_B) := \text{OT}(\mu_A, \mu_B; D_S^p)^{1/p}$.

Wasserstein Barycenters. This represents the notion of averaging in the Wasserstein space. To be precise, the Wasserstein barycenter (Agueh & Carlier, 2011) is a probability measure that minimizes the weighted sum of (p -th power) Wasserstein distances to the given N measures $\{\mu_1, \dots, \mu_N\}$, with corresponding weights $\eta = \{\eta_1, \dots, \eta_N\} \in \Sigma_N$. Hence, it can be written as

$$\mathcal{B}_p(\mu_{A_1}, \dots, \mu_{A_N}) = \arg \min_{\mu} \sum_{i=1}^N \eta_i \mathcal{W}_p(\mu, \mu_{A_i})^p. \quad (2)$$

Regularization. The solution to Eq. (1) lies at the extreme points of the polyhedra corresponding to this linear program. However, computing this exactly is expensive and scales in

about $\mathcal{O}(n^3 \log(n))$ (n being the cardinality of the support of the measures). A common way to tackle this nowadays, is to consider the entropy regularized Wasserstein distance, $\mathcal{W}_{p,\lambda}(\mu_A, \mu_B) := \text{OT}_\lambda(\mu_A, \mu_B; D_S^p)^{1/p}$, following Cuturi (2013). Here the search space for the optimal T is instead restricted to a smooth solution close to the extreme points of the linear program by subtracting $\lambda H(T)$ from the linear objective, where $H(T) = -\sum_{ij} T_{ij} \log T_{ij}$.

Sinkhorn iterations. The regularized problem is solved efficiently using Sinkhorn iterations (Sinkhorn, 1964), the cost of each iteration is $\mathcal{O}(n^2)$. However, Altschuler et al. (2017) show that convergence can be attained in a number of iterations which is independent of n , resulting in the overall complexity of $\tilde{\mathcal{O}}(n^2)$. These iterations mostly consist of matrix-vector products are further amenable to GPU acceleration.

4. Proposed Algorithm

In this section, we discuss our proposed algorithm for model aggregation. Without loss of generality, we consider that we are averaging the parameters of only two neural networks. For now, we ignore the bias parameters and we only focus on the weights. This is to make the presentation succinct, and it can be easily extended to take care of these aspects..

Motivation. As alluded to earlier in the introduction, the problem with vanilla averaging of parameters is the lack of one-to-one correspondence between the parameters of the models. In particular, for a given layer, there is no direct matching between the neurons of the two models. For e.g., this means that the k^{th} neuron of model A might behave very differently (in terms of the feature it detects) from the k^{th} neuron of the other model B, and instead might be quite similar in functionality to the $k + 1^{\text{th}}$ neuron.

Imagine, if we knew a perfect matching between the neurons, then we could simply align the neurons of (say) the model A with respect to B. Having done this, it would make more sense to perform vanilla averaging of the neuron parameters than before. The matching or assignment could be formulated as a permutation matrix, and just multiplying the parameters by this matrix would align the parameters.

But in practice, it is more likely to have soft correspondences between the neurons of the two models for a given layer, especially if their number is not the same across the two models. This is where optimal transport comes in and provides us a transport map T in the form of a soft-alignment matrix. In other words, the alignment problem can be rephrased as optimally transporting the neurons in a given layer of model A to the neurons in the same layer of model B.

General procedure. Before we go further, note that since the input layer is ordered identically for both models, we start the alignment from the second layer onwards. Addi-

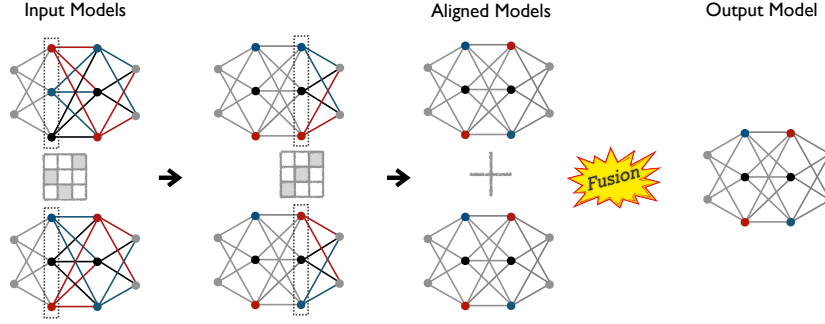


Figure 1. Model Fusion procedure: The first two steps illustrate how the model A (top) gets aligned with respect to model B (bottom). The alignment here is reflected by the ordering of the node colors in a layer. Once each layer has been aligned, the model parameters get averaged (shown by the + sign) to yield a fused model at the end.

tionally, the order of neurons for the very last layer, i.e., in the output layer, again is identical. Therefore, the (scaled) transport map at the last layer will be equal to the identity.

Now assume we are at layer ℓ . Then, we first define probability measures over neurons in this layer ℓ for the two models as, $\mu_A = (\alpha^{(\ell)}, \mathcal{S}_A[\ell])$ and $\mu_B = (\beta^{(\ell)}, \mathcal{S}_B[\ell])$, where $\mathcal{S}_A, \mathcal{S}_B$ denote the support of the measures.

Next, we use uniform distributions to initialize the histogram (or probability mass values) for each layer. Although we note that it is possible to additionally use other measures of neuron importance (Dhamdhere et al., 2019; Sundararajan et al., 2017). In particular, if the size of layer ℓ of models A and B is denoted by $n_A^{(\ell)}, n_B^{(\ell)}$ respectively, we have:

$$\alpha^{(\ell)}, \beta^{(\ell)} \leftarrow \mathbf{1}_{n_A^{(\ell)}}/n_A^{(\ell)}, \mathbf{1}_{n_B^{(\ell)}}/n_B^{(\ell)}. \quad (3)$$

Now, in terms of the alignment procedure, we first align the incoming edge weights for the current layer ℓ . This can be done by post-multiplying with the previous layer transport matrix $\mathbf{T}^{(\ell-1)}$, normalized appropriately via the inverse of the corresponding column marginals $\beta^{(\ell-1)}$:

$$\widehat{\mathbf{W}}_A^{(\ell, \ell-1)} \leftarrow \mathbf{W}_A^{(\ell, \ell-1)} \mathbf{T}^{(\ell-1)} \text{diag}\left(\frac{1}{\beta^{(\ell-1)}}\right). \quad (4)$$

This update can be interpreted as follows: the matrix $\mathbf{T}^{(\ell-1)} \text{diag}(\beta^{(\ell-1)})$ has $n_B^{(\ell-1)}$ columns in the simplex $\Sigma_{n_A^{(\ell-1)}}$, thus post-multiplying $\mathbf{W}_A^{(\ell, \ell-1)}$ with it will produce a convex combination of the points in $\mathbf{W}_A^{(\ell, \ell-1)}$ with weights defined by the optimal transport map $\mathbf{T}^{(\ell-1)}$.

Once this has been done, we focus on aligning the neurons in this layer ℓ of the two models. Let us assume, we have a suitable ground metric D_S (which we discuss in the sections ahead). Then the optimal transport map $\mathbf{T}$ between the measures μ_A, μ_B for layer ℓ can be computed as follows:

$$\mathbf{T}^{(\ell)}, \sim \leftarrow \mathcal{W}_{2, \lambda}(\mu_A, \mu_B, D_S). \quad (5)$$

The $\sim$ acts as a placeholder for the obtained Wasserstein-distance value. Now, we use this transport map $\mathbf{T}$ to align the neurons (more precisely their weights) of the first model (A) with respect to the second (B), as follows:

$$\widetilde{\mathbf{W}}_A^{(\ell, \ell-1)} \leftarrow \mathbf{T}^{(\ell)\top} \text{diag}\left(\frac{1}{\beta^{(\ell)}}\right) \widehat{\mathbf{W}}_A^{(\ell, \ell-1)}. \quad (6)$$

We will refer to the model with the weights in Eq. (6) as model A aligned with respect to model B. Hence, with this alignment in place, we can average the weights of two layers to obtain the final weight matrix $\mathbf{W}^{(\ell, \ell-1)}$, as shown ahead:

$$\mathbf{W}^{(\ell, \ell-1)} \leftarrow \frac{1}{2} \left(\widetilde{\mathbf{W}}_A^{(\ell, \ell-1)} + \mathbf{W}_B^{(\ell, \ell-1)} \right). \quad (7)$$

We carry out the described procedure over all the layers sequentially. The above discussion implies that we need to design a ground metric D_S between the inter-model neurons. So, we branch out into the following two strategies based on the nature of the ground metric used for the alignment.

4.1. Activation-based alignment

In this variant, we run inference over samples ($\mathbf{x}_i$) in the training set (say, m of them) and store the activations for all neurons in the model, as shown in Eq. (8). Then the neurons across the two models are considered to be similar if they produce similar activation outputs for the given set of samples. We measure this by computing the Euclidean distance between the resulting vector of activations. This serves as the ground metric for optimal transport computations.

$$\mathcal{S}_A[2, \dots, L][i], \mathcal{S}_B[2, \dots, L][i] \leftarrow M_A(\mathbf{x}_i), M_B(\mathbf{x}_i). \quad (8)$$

In practice, we use the pre-activations, i.e., the values obtained before applying an activation function such as ReLU or Sigmoid. Lastly, a complete description of the overall procedure with this kind of ground metric is summarized in Algorithm 1, when the alignment type $\psi = \text{'acts'}$.

Algorithm 1. Model Fusion (with $\psi = \{\text{'acts'}, \text{'wts'}\}$ —alignment)

- 1: **hyperparameters:** OT regularization λ . # training samples m to compute activations (if $\psi = \text{'acts'}$)
- 2: **input:**
Trained models M_A, M_B , with same #layers L
- 3: **notation:** Size of layer j of model M_A is written as $n_A^{(j)}$, and the weight matrix between the layer ℓ and $\ell-1$ is denoted as $\mathbf{W}_A^{(\ell, \ell-1)}$ for model M_A (similarly for M_B). Neuron support tensors are written as $\mathbf{S}_A, \mathbf{S}_B$.
- 4: **initialize** For the input layer, $n_A^{(1)} = n_B^{(1)} \leftarrow n^{(1)}$; $\boldsymbol{\alpha}^{(1)} = \boldsymbol{\beta}^{(1)} \leftarrow \mathbf{1}_{n^{(1)}}/n^{(1)}$ and so the transport map is defined as $\mathbf{T}^{(1)} \leftarrow \text{diag}(\boldsymbol{\beta}^{(1)}) \mathcal{I}_{n^{(1)} \times n^{(1)}}$.
- 5: **for** each layer $\ell = 2, \dots, L$ **do**
 - ▷ **Align incoming edges for M_A weights**
 - 6: $\widehat{\mathbf{W}}_A^{(\ell, \ell-1)} \leftarrow \mathbf{W}_A^{(\ell, \ell-1)} \mathbf{T}^{(\ell-1)} \text{diag}(\frac{1}{\beta^{(\ell-1)}})$
 - ▷ **Initialize probability mass values for neurons**
 - 7: $\boldsymbol{\alpha}^{(\ell)}, \boldsymbol{\beta}^{(\ell)} \leftarrow \mathbf{1}_{n_A^{(\ell)}}/n_A^{(\ell)}, \mathbf{1}_{n_B^{(\ell)}}/n_B^{(\ell)}$
 - ▷ **Get support for the measures, c.f., Eqs. (8), (9)**
 - 8: $\mathbf{S}_A[\ell], \mathbf{S}_B[\ell] \leftarrow \text{GETSUPPORT}(M_A, M_B, \psi, \ell)$
 - ▷ **Define probability measures**
 - 9: $\mu_A, \mu_B \leftarrow (\boldsymbol{\alpha}^{(\ell)}, \mathbf{S}_A[\ell]), (\boldsymbol{\beta}^{(\ell)}, \mathbf{S}_B[\ell])$
 - ▷ **Form ground metric ($\forall p \in [n_A^{(\ell)}], q \in [n_B^{(\ell)}]$)**
 - 10: $D_S[p, q] \leftarrow \|\mathbf{S}_A[\ell][p] - \mathbf{S}_B[\ell][q]\|_2$
 - ▷ **Compute OT map and distance**
 - 11: $\mathbf{T}^{(\ell)}, \sim \leftarrow \mathcal{W}_{2, \lambda}(\mu_A, \mu_B, D_S)$
 - ▷ **Align model M_A neurons**
 - 12: $\widetilde{\mathbf{W}}_A^{(\ell, \ell-1)} \leftarrow \mathbf{T}^{(\ell)\top} \text{diag}(\frac{1}{\beta^{(\ell)}}) \widehat{\mathbf{W}}_A^{(\ell, \ell-1)}$
 - ▷ **Average model weights**
 - 13: $\mathbf{W}^{(\ell, \ell-1)} \leftarrow \frac{1}{2}(\widetilde{\mathbf{W}}_A^{(\ell, \ell-1)} + \mathbf{W}_B^{(\ell, \ell-1)})$
 - 14: **end for**

4.2. Weight-based alignment

Here, we consider that the support of each neuron is given by the weights of the incoming edges (stacked in a vector). Thus, a neuron can be thought as being represented by the row corresponding to it in the weight matrix. As indicated in Eq. (9), this forms the support when $\psi = \text{'wts'}$. The reasoning for such a choice stems from the neuron activation at a particular layer being calculated as the inner product between this weight vector and the previous layer output. The ground metric then used for OT computations is again the Euclidean distance between weight vectors corresponding to the neurons p of M_A and q of M_B (see LINE 10 of Algorithm 1). Besides this difference of employing the actual weights in the ground metric (LINE 8), rest of the procedure remains identical.

$$\mathbf{S}_A[\ell], \mathbf{S}_B[\ell] \leftarrow \widehat{\mathbf{W}}_A^{(\ell, \ell-1)}, \mathbf{W}_B^{(\ell, \ell-1)}. \quad (9)$$

For the weight-based alignment, the update step in Eq. (4) bears resemblance to barycentric projection as well as the local quadratic approximation used in the derivation of the free-support barycenter (Cuturi & Doucet, 2014), and is discussed in Appendix S7.

4.3. Discussion

Pros and cons of alignment type. An advantage of the weight-based alignment is that it is independent of the dataset samples, making it useful in privacy-constrained scenarios. Since the activation-based alignment only needs unlabeled data, an interesting prospect for a future study would be to utilize synthetic data. On the flip side, activation-based alignment can even be used when the model layers are of unequal size (see Section 6.2), a setting where the current weight-based alignment by itself would be inadequate.

Practical considerations. (a) We flatten the convolutional layers for ground metric computations and hence all the weights of a 3D (# channels $\times$ width $\times$ height) convolution block are aligned with the same transport map. (b) We find it useful to normalize the weight or activation vectors to unit norm while computing the ground metric.

Unsuitability of fixed-support barycenters. The formulation of layer-wise averaging is similar to that of the Wasserstein barycenters of the probability measures defined over the individual model layers. But, it is not possible to directly employ the commonly used Iterative Bregman Projections (IBP) scheme (Benamou et al., 2015). This is because it is based on the fixed-support barycenter assumption, which does not make sense in our case where the supports are given by vector of activations or incoming weights.

Averaging more than 2 models. A straightforward way is by calling this method of averaging two models recursively as a helper routine. When using the weight-based alignment, it can be done more naturally, by using the free-support Wasserstein barycenters algorithm from (Cuturi & Doucet, 2014).¹ But, in the case of two models, we stick to the methodology used in the Algorithm 1 to utilize the computed transport map, as it leads to a simpler algorithm.

5. Experiments

Datasets and Models. We first study the empirical performance of the proposed method for averaging parameters of a multi-layer perceptron (MLP) on the MNIST handwritten digits dataset. The particular network that we consider has 3

¹In fact, this method would also give the flexibility to choose the size of the support of the barycenter. This implies that it is possible to aggregate into a single layer which is either smaller or bigger in size as compared to the individual model layers.

fully-connected hidden layers comprising of 400, 200, 100 units respectively. The input is a 28×28 image and the output is a vector of classification scores of length 10. We call this network MLPNET and the overall architecture is $784 \rightarrow 400 \rightarrow 200 \rightarrow 100 \rightarrow 10$. Further, we benchmark on the CIFAR10 dataset, and for which we consider the VGG11 (Simonyan & Zisserman, 2014) convolutional network and the ResNet18 residual network (He et al., 2016).

Baselines. In order to compare the model fusion performance, we mention the performance of ‘prediction’ ensembling and ‘vanilla’ averaging, besides the performance of individual models. Prediction ensembling refers to keeping all the models and averaging their predictions (output layer scores). It thus reflects in some sense the ideal performance that we can hope to achieve with a single model. Vanilla averaging denotes the direct averaging of parameters without taking into account the underlying structure. All the performance scores are test accuracies. Full experimental details are provided in Appendix S1.1.

5.1. Results: Fully-connected networks

Here, the individual models used are MLPNET’s which have been trained for 10 epochs on MNIST. They differ only in their seeds and thus in the initialization of the parameters alone. We ensemble the final checkpoint of these models via OT averaging and the baseline methods.

M_A	M_B	Prediction avg.	Vanilla avg.	m	OT avg. (Sinkhorn) Accuracy (mean $\pm$ stdev)	M_A aligned
(a) Activation-based Alignment						
97.72	97.75	97.88	73.84	2	24.80 ± 6.93	20.08 ± 2.42
				10	75.04 ± 11.35	88.18 ± 8.45
				25	90.95 ± 3.98	95.36 ± 0.96
				50	93.47 ± 1.69	96.04 ± 0.59
				100	95.40 ± 0.52	97.05 ± 0.17
				200	95.78 ± 0.52	97.01 ± 0.16
(b) Weight-based Alignment						
97.72	97.75	97.88	73.84	—	95.66	96.32

Table 1. One-shot model averaging on MNIST with MLPNET. Results showing the performance (i.e., test classification accuracy (in %)) of the OT averaging in contrast to the baseline methods. The last column refers to the aligned model A which gets (vanilla) averaged with model B, giving rise to our OT averaged model. m is the size of mini-batch over which activations are computed.

Activation-based alignment: This variant of OT averaging involves computing activations for input examples of batch size m , which are randomly sampled from the training dataset. Thus, we report the mean and standard deviation (stdev) of accuracies over 5 different seeds, for each value of m . Table 1 (a) shows the results for this setting.

As the batch size increases, the performance of OT averaging keeps increasing until it starts to plateau around

$m = 200$. Besides, the standard deviation also goes down with an increase in the batch size. From ≈ 25 examples, the OT averaging outperforms the vanilla averaging with ease.

Weight-based alignment: In Table 1 (b), we show its performance on the similar setting of MLPNET on MNIST. The results are similar to those for the activation-based alignment, albeit slightly lower.

In the above tables, we observe that OT averaging significantly outperforms the vanilla averaging. Yet, these results are still slightly lower than the individual models and the ideal performance target from prediction ensembling. Nevertheless, we discuss a simpler way around in Section 5.4.

5.2. Ablation studies

Exact OT and runtime efficiency: From our discussion in Section 3, running the Sinkhorn algorithm for OT would be roughly quadratic in the width of the network (which is ≤ 600 for the ones considered here). But, this is not an issue as networks are typically deeper than wide and the cost is only linear in the number of layers. In fact, our fusion procedure is efficient enough for deep neural networks that we mostly utilize the exact OT solver. As evident from Table 2, the exact OT computations lead to an improved test accuracy, in comparison to when employing Sinkhorn. Switching to exact OT barely hurts the running time; for e.g., it takes ≤ 5 seconds on 1 Nvidia V100 GPU to fuse two VGG11 models (c.f. Section S1.4 for more details). Our presented results will henceforth be based on exact OT.

M_A	M_B	Prediction avg.	Vanilla avg.	Alignment Type	OT avg. Accuracy (mean)	M_A aligned
Regularized OT (via Sinkhorn)				Activation	95.78	97.01
97.72	97.75	97.78	73.84	Weight	95.66	96.32
Exact OT				Activation	96.21	97.72
97.72	97.75	97.78	73.84	Weight	96.63	97.72

Table 2. Exact vs Regularized OT: Results showing the performance gain with exact OT for activation/weight based alignment.

Effect of layer width: Table S7 in the appendix illustrates that as the width of networks increases, the gap in performance of one-shot OT averaging compared to the best individual network decreases. This also suggests a very interesting potential connection with the mean-field limit for neural networks (Mei et al., 2019). Here, the authors show that as the size of the hidden layer goes to infinity, doing gradient descent on the network weights is equivalent to considering a probability density over the neurons in a layer, which evolves with Wasserstein gradient flow. Then given two neural networks evolving under the dynamics of Wasserstein gradient flow, fusing them into one network by Wasserstein barycenter would be a natural consideration. We refer to Appendix S2 for additional ablation studies.

5.3. Results: convolutional and residual networks

Table 3 shows the more interesting setting of VGG11 and RESNET18 on CIFAR10. Here, the two individual models (differing only in their seeds) are trained on CIFAR10 for 300 epochs, with training details mentioned in S1.1. Vanilla averaging absolutely fails in this case, and is $3\text{--}5\times$ worse than OT averaging, in case of RESNET18 and VGG11 respectively. In contrast to MLPNET which has only 3 hidden layers, the VGG11 and RESNET18 models used here have 9 and 18 hidden layers respectively. Also with many more parameters. Hence as the models get deeper, the misalignment between the parameters gets more severe, indicating the importance of alignment via OT before averaging.

Dataset + Model	M_A	M_B	Prediction avg.	Vanilla avg.	OT avg.	Finetuning Vanilla	OT
MNIST+MLPNET	97.72	97.75	97.78	73.84	96.54	<u>98.23</u>	98.35
CIFAR10+VGG11	90.31	90.50	91.34	17.02	85.98	90.39	<u>90.73</u>
CIFAR10+RESNET18	93.11	93.20	93.89	18.49	67.46	93.49	<u>93.78</u>

Table 3. Results for convolutional and residual networks, as well as for MLP, along with the effect of finetuning the fused models.

5.4. Finetuning from fused models

While the alignment of parameters reduces the non-convexity which arises from permutation symmetries, a lot of non-convexity in the optimization landscape still remains around. Our hope is that the OT average serves as a good starting point for further finetuning (i.e., retraining), and results in a net performance gain.

We evaluate the finetuning performance of the models whose weights have been aggregated via vanilla and OT averaging, at several finetuning learning rates and schedules (the detailed results are located in Appendix S3). In Table 3, we present the best obtained results for each of the averaging methods. This retraining takes place for a total of 60, 100, and 120 epochs in case of MLPNET, VGG11, and RESNET18 respectively. Some typical retraining curves, i.e., Figures S4 and S5, can be found in Appendix S4.

We observe that for both vanilla and OT averaging it helps to retrain, but in comparison, the OT averaging results in a better score for all the settings in Table 3. For e.g., in case of RESNET18, with finetuning the performance of OT average gets almost close to that of prediction ensembling, despite having a single model (and not the best way² to handle skip connections in OT average). We also considered finetuning

²The down-sampling skip connection in RESNET18 also requires alignment and results in a transport map, besides the one coming from the residual part. For now, we simply average these two transport maps, but we plan to study better ways in the future.

the individual models across the different hyperparameter settings (which of course will be infeasible in practice), but the best accuracy mustered via this attempt was 93.51, in comparison to 93.78 obtained with OT average + finetuning. Overall, this shows that retraining of the fused models alleviates the gap in performance remaining after one-shot OT averaging, and also outperforms the best individual models.

6. Applications

6.1. One-shot skill transfer

We again consider the setting of merging two models A and B but now assume that model A has some special skill or capability which B does not possess. However, B is overall more powerful on the remaining set of skills in comparison to A. The goal of fusion now is to obtain a single model that can gain from the strength of B on overall skills and also acquire the specialized skill possessed by A.

Such a scenario can arise e.g. in reinforcement learning where these models are agents that have had different training experiences so far. Another use case lies in federated learning, where model A is a client application that has been trained to perform well on certain personalized tasks and model B is the server that typically has a strong skill set for a wider range of tasks.

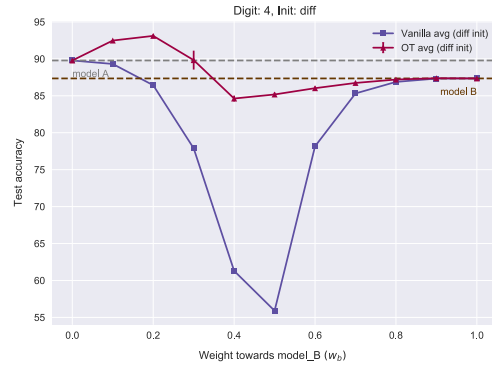


Figure 2. **One-shot skill transfer performance:** Each point for OT avg. curve (magenta colored) is obtained by activation-based alignment with a batch size $m = 400$, and we plot the mean performance over 5 seeds along with the error bars, which show the corresponding standard deviation. *No retraining is done here.*

An important constraint in such settings is that training examples can not be shared between A and B so as to maintain privacy.³ Hence, we approach this problem by fusing the given models via the OT averaging.

More concretely, we consider the MNIST digit classification task, where the unique skill of model A corresponds to the

³Only A's training samples are used for computing the activations, avoiding the mentioned sharing of data.

ability to predict one particular ‘personalized’ label (say 4) which is unseen for B. Model B contains 90% of the remaining training set (i.e., excluding the label 4), while A has the other 10%. Both the models are trained on their portions of the data for 10 epochs each. Except for their different parameter initializations, all other settings are identical. On their local test sets, which are split analogously to the training sets, the two models achieve 96.82% and 97.98% accuracy respectively. Whereas on the overall (global) test set, they obtain 89.78% and 87.35% accuracy respectively.

Fig. 2 demonstrates how OT model fusion (without any retraining) significantly outperforms vanilla averaging in terms of the overall test accuracy, as the weight w_B towards model B varies between 0 and 1. More interestingly, for the $w_B = 0.2$, OT avg. achieves 93.11% overall test set accuracy, which surpasses the performance of *both* individual models by $\geq 3\%$, confirming the successful skill transfer from both parent models. Our obtained experimental results are robust to other scenarios when (a) some other label (say 6) serves as the special skill, (b) models have the same initialization of parameters and (c) the % of remaining data split is different. Appendix S5 collects these results.

6.2. Fusion under size constraints

An advantage of our method is that we do not require the layer widths to be the same for the input models. Such a constraint can arise in federated learning, as the storage capability of each client might be different. This scenario of having different layer widths across the models rules out the use of vanilla averaging of parameters, a core component of the widely prevalent, FedAvg (McMahan et al., 2016).

Dataset + Model	# params (M_A, M_B)	M_A	M_B	OT avg.	Finetuning M_B OT avg.
MNIST+	(414 K, 182 K)	98.11	97.84	95.67	98.06
MLPNET	(414 K, 32 K)	98.11	97.08	96.50	97.31
CIFAR10+	(118 M, 32 M)	91.22	90.66	86.73	90.67
VGG11	(118 M, 3 M)	91.22	89.38	88.40	89.64
					98.22
					97.42
					90.89
					89.85

Table 4. Compressing M_A to smaller models. The finetuning results of each method are at their best scores across different finetuning hyperparameters (like, learning rate schedules). OT avg. has the same number of parameters as M_B .

Model compression. We present the results for such a setting in Table 4, where all the hidden layers of model M_A , are a constant $\rho \times$ wider than all the hidden layers of model M_B . On MNIST + MLPNET, $\rho \in \{2, 10\}$, whereas for CIFAR10 + VGG11, $\rho \in \{2, 8\}$. This translates into the four rows present in the Table 4. As a first baseline, we consider the performance of finetuning the model M_B . We observe that across all the settings, OT avg. + finetuning outperforms this baseline as well as the original model M_B .

(See Appendix S9 for detailed results). In general, this strategy can act as a potentially useful post-processing step when compressing models by structured pruning methods, or during the training itself as a regularizer (see Appendix S2.4).

M_A	M_B	OT avg.	Finetuning M_B OT avg.	Distillation Random M_B OT avg.
98.11	97.84	95.49	98.04 98.19	98.18 98.22 98.30
Mean across distillation temperatures				98.13 98.17 98.26

Table 5. Compressing the bigger teacher model M_A to half its size. The distillation scores for each student network initialization are taken for its best hyperparameter values. Both finetuning and distillation were run for 60 epochs using SGD with the same hyperparameters. Each entry has been averaged across 4 seeds.

Distillation. As another alternative option, we consider distillation, and investigate its performance compared to OT and other averaging schemes. We note that distillation requires sharing the student’s training data with the teacher network, which might not be possible in all scenarios. The results of these experiments are reported in Table 5.

We initialize the student model for distillation in 3 possible ways: random, OT avg., and model M_B . Here, we focus on MNIST + MLPNET, as this allows us to perform an extensive sweep over the distillation based hyperparameters (temperature, loss-weighting factor) for each method. The best scores for initializing with OT avg. are significantly better than the best scores for either random or model M_B based initializations.

Further, when averaged across the considered temperature values = $\{20, 10, 8, 4, 1\}$, we observe that distillation of M_A into random or M_B -based initializations perform worse than simple OT avg. + finetuning (which also doesn’t require doing such a sweep that might become prohibitive for larger models/datasets). Results for the width-ratio $\rho = 10$ for MNIST + MLPNET as well as detailed results can be found in Appendix S10.

Overall, this points that in cases where distillation is feasible, initializing by OT avg. can be helpful. If however, circumstances do not permit this, OT avg. + finetuning can go a long way in transferring the knowledge from a bigger model into a smaller one.

One-shot skill transfer. Results for this use-case from Section 6.1, under constraints that necessitate different layer widths for models A & B, can be found in the Appendix S6.

7. Conclusion

We show that averaging the weights of models, by first doing a layer-wise (soft) alignment of the neurons via optimal transport, significantly outperforms the vanilla averaging of weights. Furthermore, we show a successful one-shot trans-

fer of knowledge between models without sharing training data, which is not possible with other techniques to the best of our knowledge. In general, the OT average when further finetuned, allows for just keeping one model rather than a complete ensemble of models at inference.

Future avenues include application in distributed optimization and continual learning, besides extending the current fusion toolkit to handle models with different number of layers, as well as fusing recurrent and generative models. The promising empirical results of the presented algorithm, thus warrant attention for further investigation and use-cases.

Acknowledgments

We would like to thank Rémi Flamary, Boris Muzellec, Sebastian Stich and other members of MLO, as well as the anonymous reviewers for their comments and feedback.

References

- Agueh, M. and Carlier, G. Barycenters in the wasserstein space. *SIAM Journal on Mathematical Analysis*, 43(2):904–924, 2011. 1, 3
- Altschuler, J., Weed, J., and Rigollet, P. Near-linear time approximation algorithms for optimal transport via sinkhorn iteration. In *Advances in Neural Information Processing Systems*, pp. 1964–1974, 2017. 3
- Ambrosio, L., Gigli, N., and Savare, G. Gradient flows: in metric spaces and in the space of probability measures. 2006. URL <https://www.springer.com/gp/book/9783764387211>. 21
- Araújo, D., Oliveira, R. I., and Yukimura, D. A mean-field limit for certain deep neural networks, 2019. 22
- Backhoff-Veraguas, J., Fontbona, J., Rios, G., and Tobar, F. Bayesian learning with wasserstein barycenters. *arXiv preprint arXiv:1805.10833*, 2018. 2
- Benamou, J.-D., Carlier, G., Cuturi, M., Nenna, L., and Peyré, G. Iterative bregman projections for regularized transportation problems. *SIAM Journal on Scientific Computing*, 37(2):A1111–A1138, 2015. 5
- Breiman, L. Bagging predictors. *Machine Learning*, 24(2): 123–140, Aug 1996. ISSN 1573-0565. doi: 10.1023/A:1018054314350. URL <https://doi.org/10.1023/A:1018054314350>. 2
- Bucilă, C., Caruana, R., and Niculescu-Mizil, A. Model compression. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’06, pp. 535–541, New York, NY, USA, 2006. ACM. ISBN 1-59593-339-5. doi: 10.1145/1150402.1150464. URL <http://doi.acm.org/10.1145/1150402.1150464>. 2
- Cuturi, M. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in neural information processing systems*, pp. 2292–2300, 2013. 2, 3
- Cuturi, M. and Doucet, A. Fast computation of wasserstein barycenters. In Xing, E. P. and Jebara, T. (eds.), *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pp. 685–693, Beijing, China, 22–24 Jun 2014. PMLR. 1, 2, 5, 21
- Dhamdhere, K., Sundararajan, M., and Yan, Q. How important is a neuron. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SylKoo0cKm>. 4
- Dognin, P., Melnyk, I., Mroueh, Y., Ross, J., Santos, C. D., and Sercu, T. Wasserstein barycenter model ensembling. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=H1g4k309F7>. 2
- Ferradans, S., Papadakis, N., Rabin, J., Peyré, G., and Aujol, J.-F. Regularized discrete optimal transport. *Scale Space and Variational Methods in Computer Vision*, pp. 428–439, 2013. ISSN 1611-3349. doi: 10.1007/978-3-642-38267-3_36. URL http://dx.doi.org/10.1007/978-3-642-38267-3_36. 21
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial nets. In *Advances in neural information processing systems*, pp. 2672–2680, 2014. 2
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun 2016. doi: 10.1109/cvpr.2016.90. URL <http://dx.doi.org/10.1109/CVPR.2016.90>. 2, 6
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 2, 25
- Kantorovich, L. V. On the translocation of masses. In *Dokl. Akad. Nauk. USSR (NS)*, volume 37, pp. 199–201, 1942. 1, 21
- Leontev, M. I., Islenteva, V., and Sukhov, S. V. Non-iterative knowledge fusion in deep convolutional neural networks. *arXiv preprint arXiv:1809.09399*, 2018. 2
- McMahan, H. B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data, 2016. 8
- Mei, S., Misiakiewicz, T., and Montanari, A. Mean-field theory of two-layers neural networks: dimension-free bounds and kernel limit, 2019. 6, 22
- Monge, G. Mémoire sur la théorie des déblais et des remblais. *Histoire de l’Académie Royale des Sciences de Paris*, 1781. 1, 21
- Morcos, A. S., Raghu, M., and Bengio, S. Insights on representational similarity in neural networks with canonical correlation, 2018. 14
- Schapire, R. E. A brief introduction to boosting. In *Proceedings of the 16th International Joint Conference on Artificial Intelligence - Volume 2*, IJCAI’99, pp. 1401–1406, San Francisco, CA, USA, 1999. Morgan Kaufmann Publishers Inc. URL <http://dl.acm.org/citation.cfm?id=1624312.1624417>. 2

- Schmidhuber, J. Learning complex, extended sequences using the principle of history compression. *Neural Computation*, 4(2): 234–242, 1992. 2
- Shen, Z., He, Z., and Xue, X. Meal: Multi-model ensemble via adversarial learning, 2018. 2
- Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition, 2014. 2, 6
- Singh, S. P., Hug, A., Dieuleveut, A., and Jaggi, M. Context mover’s distance & barycenters: Optimal transport of contexts for building representations, 2018. 2
- Sinkhorn, R. A relationship between arbitrary positive matrices and doubly stochastic matrices. *The Annals of Mathematical Statistics*, 35(2):876–879, 1964. ISSN 00034851. 3
- Smith, J. and Gashler, M. An investigation of how neural networks learn from the experiences of peers through periodic weight averaging. In *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 731–736. IEEE, 2017. 2
- Solomon, J., de Goes, F., Peyré, G., Cuturi, M., Butscher, A., Nguyen, A., Du, T., and Guibas, L. Convolutional wasserstein distances: Efficient optimal transportation on geometric domains. *ACM Trans. Graph.*, 34(4):66:1–66:11, July 2015. ISSN 0730-0301. doi: 10.1145/2766963. URL <http://doi.acm.org/10.1145/2766963>. 2
- Stich, S. U. Local sgd converges fast and communicates little. In *ICLR 2019 - International Conference on Learning Representations*, 2019. 2
- Sundararajan, M., Taly, A., and Yan, Q. Axiomatic attribution for deep networks, 2017. 4
- Utans, J. Weight averaging for neural networks and local resampling schemes. In *Proc. AAAI-96 Workshop on Integrating Multiple Learned Models*. AAAI Press, pp. 133–138, 1996. 2
- Wang, H., Yurochkin, M., Sun, Y., Papailiopoulos, D., and Khazaeni, Y. Federated learning with matched averaging. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=BkluqlSFDS>. 2
- Wolpert, D. H. Original contribution: Stacked generalization. *Neural Netw.*, 5(2):241–259, February 1992. ISSN 0893-6080. doi: 10.1016/S0893-6080(05)80023-1. URL [http://dx.doi.org/10.1016/S0893-6080\(05\)80023-1](http://dx.doi.org/10.1016/S0893-6080(05)80023-1). 2
- Yurochkin, M., Agarwal, M., Ghosh, S., Greenewald, K., Hoang, T. N., and Khazaeni, Y. Bayesian nonparametric federated learning of neural networks, 2019. 2

Appendix

S1. Technical specifications

Before going further, note that our code will be made available under the following link <https://github.com/modelfusion>.

S1.1. Experimental Details

VGG11 training details. It is trained by SGD for 300 epochs with an initial learning rate of 0.05, which gets decayed by a factor of 2 after every 30 epochs. Momentum = 0.9 and weight decay = 0.0005. The batch size used is 128. Checkpointing is done after every epoch and the best performing checkpoint in terms of test accuracy is used as the individual model. The block diagram of VGG11 architecture is shown below for reference.

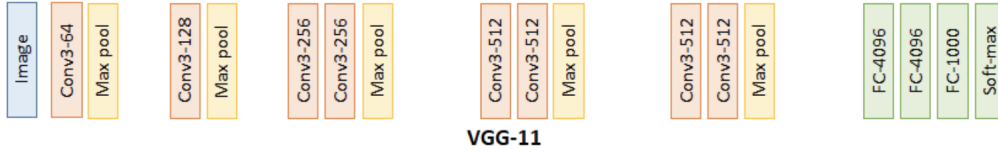


Figure S1. Block diagram of the VGG11 architecture. Adapted from <https://bit.ly/2ksX5Eq>.

MLPNET training details. This is also trained by SGD at a constant learning rate of 0.01 and momentum = 0.5. The batch size used is 64.

RESNET18 training details. Again, we use SGD as the optimizer, with an initial learning rate of 0.1, which gets decayed by a factor of 10 at epochs {150, 250}. In total, we train for 300 epochs and similar to the VGG11 setting we use the best performing checkpoint as the individual model. Other than that, momentum = 0.9, weight decay = 0.0001, and batch size = 256. We skip the batch normalization for the current experiments, however, it can possibly be handled by simply multiplying the batch normalization parameters in a layer by the obtained transport map while aligning the neurons.

Other details. *Pre-activations.* The results for the activation-based alignment experiments are based on pre-activation values, which were generally found to perform slightly better than post-activation values.

Regularization. The regularization constant used for the activation-based alignment results in Table 1 is 0.05.

Common details. The bias of a neuron is set to zero in all of the experiments. It is possible to handle it as a regular weight by keeping the corresponding input as 1, but we leave that for future work.

S1.2. Combining weights and activations for alignment

The output activation of a neuron over input examples gives a good signal about the presence of features in which the neuron gets activated. Hence, one way to combine this information in the above variant with weight-based alignment is to use them in the probability mass values.

In particular, we can take a mini-batch of samples and store the activations of all the neurons. Then we can use the mean activation as a measure of a neuron’s significance. But it might be that some neurons produce very high activations (in absolute terms) irrespective of the kind of input examples. Hence, it might make sense to also look at the standard deviation of activations. Thus, one can combine both these factors into an importance weight for the neuron as follows:

$$\text{importance}_k[2, \dots, L] = \overline{M}_k([x_1, \dots, x_d]) \odot \sigma(M_k([x_1, \dots, x_d])) \quad (10)$$

Here, M_k denotes the k^{th} model into which we pass the inputs $[x_1, \dots, x_d]$, $\overline{M}$ denotes the mean, $\sigma(\cdot)$ denotes the standard

deviation and $\odot$ denotes the elementwise product. Thus, we can now set the probability mass values $b_k^{(l)} \propto \text{importance}_k[l]$, and the rest of the algorithm remains the same.

S1.3. Optimal Transport

We make use of the Python Optimal Transport (POT)^{S1} for performing the computation of Wasserstein distances and barycenters on CPU. These can also be implemented on the GPU to further boost the efficiency, although it suffices to run on CPU for now, as evident from the timings below.

S1.4. Timing information

The following timing benchmarks are done on 1 Nvidia V100 GPU. The time taken to average two MLPNET models for MNIST is ≈ 3 seconds. For averaging VGG11 models on CIFAR10, it takes about ≈ 5 seconds. While in case of RESNET18 on CIFAR10, it takes ≈ 7 seconds. These numbers are for the activation-based alignment, and also include the time taken to compute the activations over the mini-batch of examples.

The weight-based alignment can be faster as it does not need to compute the activations. For instance, when weight-based alignment is employed to average two VGG11 models on CIFAR10, it takes ≈ 2.5 seconds.

S2. Ablation studies

S2.1. Aggregation performance as training progresses

We compare the performance of averaged models at various points during the course of training the individual models (for the setting of MLPNet on MNIST). We notice that in the early stages of training, vanilla averaging performs even worse, which is not the case for OT averaging. The corresponding Figure S2 and Table S1 can be found in Section S2.1 of the Appendix. Overall, we see OT averaging outperforms vanilla averaging by a large margin, thus pointing towards the benefit of aligning the neurons via optimal transport.

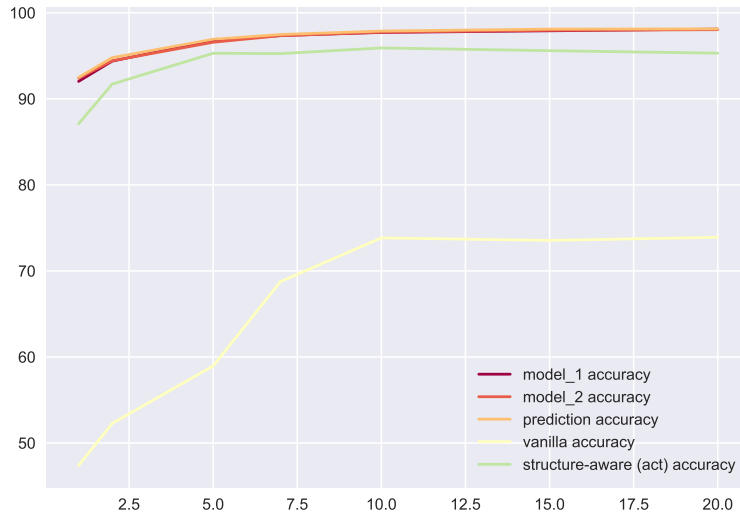


Figure S2. Illustrates the performance of various aggregation methods as training proceeds, for (MNIST, MLPNET). The plots correspond to the results reported in Table S1. The activation-based alignment of the OT average is used based on $m = 200$ samples.

^{S1}<https://pot.readthedocs.io/en/stable/>

Model Fusion via Optimal Transport

<i>Epoch</i>	<i>Model A</i>	<i>Model B</i>	<i>Prediction avg.</i>	<i>Vanilla avg.</i>	<i>OT avg.</i>
01	92.03	92.40	92.50	47.39	87.10
02	94.39	94.43	94.79	52.28	91.72
05	96.83	96.58	96.93	58.96	95.30
07	97.36	97.34	97.48	68.76	95.26
10	97.72	97.75	97.88	73.84	95.92
15	97.91	97.97	98.11	73.55	95.60
20	98.11	98.04	98.13	73.91	95.31

Table S1. Activation-based alignment (MNIST, MLPNet): Comparison of performance when ensembled after different training epochs. The # samples used for activation-based alignment, $m = 50$. The corresponding plot for this table is illustrated in Figure S2.

S2.2. Transport map for the output layer.

Since our algorithm runs until the output layer, we inspect the alignment computed for the last output layer. We find that the ratio of the trace to the sum for this last transport map is ≈ 1 , indicating accurate alignment as the ordering of output units is the same across models.

S2.3. Effect of regularization

The results for activation-based alignment presented in the Table 1 in the main text use the regularization constant $\lambda = 0.05$. Below, we also show the results with a higher regularization constant $\lambda = 0.1$. As expected, we find that using a lower value of regularization constant leads to better results in general, since it better approximates OT.

M_A	M_B	<i>Prediction</i>	<i>Vanilla</i>	m	<i>OT avg.</i> Accuracy (mean $\pm$ stdev)	$M_{Aaligned}$
97.72	97.75	97.88	73.84	2	25.05 ± 7.22	19.42 ± 2.28
				10	72.86 ± 11.93	74.35 ± 14.40
				25	89.49 ± 5.21	90.88 ± 4.91
				50	92.88 ± 2.03	94.54 ± 1.36
				100	95.14 ± 0.49	96.42 ± 0.39
				200	95.70 ± 0.54	96.63 ± 0.23

Table S2. Activation-based alignment (MNIST, MLPNet): Results showing the performance (i.e., test classification accuracy) of the averaged and aligned models of OT based averaging in contrast to vanilla averaging of weights as well as the prediction based ensembling. m denotes the number of samples over which activations are computed, i.e., the mini-batch size.

However, since using the exact optimal transport is fast enough, we default to using it hereafter.

S2.4. Layer-wise Optimal Transport distances

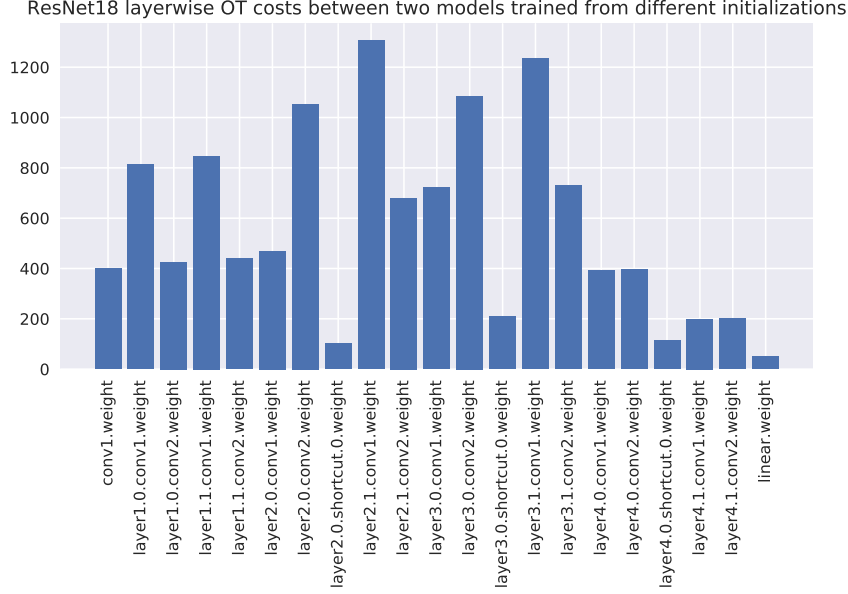


Figure S3. Illustrates the layerwise Optimal Transport costs between the corresponding layers of two ResNet18 models trained from different initializations, when using activation-based alignment with mini-batch size $m = 200$.

A possible application of our model fusion approach can be for inspecting the similarity of representations at various layers across different neural networks. Thus, it could provide an alternative perspective for this problem of understanding the similarity of representations, besides the canonical correlation analysis (CCA) based methods used in the past (Morcos et al., 2018). Figure S3 gives an example of this for two ResNet18 models trained from different initializations. Here, we used activation-based alignment with mini-batch size $m = 200$.

Another related use-case in which our algorithm can be applied is as a regularizer for training a student network, given another teacher network. So basically instead of directly computing the OT average of the layers (as done in Section 6.2), the OT cost computed as an intermediate can act as a regularization penalty. This would allow to carry out knowledge distillation with respect to the hidden layers. An extensive study, however, remains beyond the scope of this paper.

S3. Detailed finetuning results

In Tables S3, S5, and S6, we report the results of finetuning (i.e. retraining) the averaged models for (MNIST, MLPNET) and (CIFAR10, VGG11). For comparison, we also show the performance of individual models when further finetuned in this setting. Although in general, **the individual model finetuning is not realistic**, since it is not known which one will lead to an improvement and this incurs $\# \text{ models} \times \text{the finetuning cost}$.

S3.1. For MNIST + MLPNET

The finetuning is carried out for 60 epochs at the following set of constant learning rates $\{0.01, 0.002, 0.001, 0.00067, 0.0005\}$. Note that the original models were trained for 10 epochs at a learning rate of 0.01. For OT average, we use the activation-based alignment with mini-batch size $m = 200$.

Table S3 shows the results for each method at their best respective finetuning runs.

Finetuning LR	Model A	Model B	Vanilla avg.	OT avg. (exact)
<i>Baseline Results</i>				
—	97.72	97.75	73.84	96.54
<i>Results for the best finetuning run (reported at the best checkpoint)</i>				
0.01	98.21	98.13	98.23	98.35
0.002	98.13	98.03	98.13	98.21
0.001	98.09	98.03	97.98	98.14
0.00067	98.11	98.00	97.83	98.07
0.0005	98.09	98.01	97.70	98.05

Table S3. Effect of finetuning the individual and averaged models for (MNIST, MLPNet): Best finetuning runs have been reported for each method. Cells in orange highlight the best scores in each regime.

We also show in Table S4 the results when averaged across 5 finetuning runs for each of the finetuning LR, as the cost of finetuning here is not as prohibitive in comparison to finetuning VGG11 and ResNet18 models. We see that performance trend remains in accordance with the previous Table S3.

Finetuning LR	Model A	Model B	Vanilla avg.	OT avg. (exact)
<i>Baseline Results</i>				
—	97.72	97.75	73.84	96.21 ± 0.36
<i>Averaged results across the finetuning runs (reported at the best checkpoint)</i>				
0.01	98.19 ± 0.02	98.11 ± 0.02	98.22 ± 0.02	98.28 ± 0.05
0.002	98.13 ± 0.01	98.03 ± 0.01	98.13 ± 0.01	98.15 ± 0.07
0.001	98.11 ± 0.02	98.01 ± 0.01	97.99 ± 0.01	98.08 ± 0.05
0.00067	98.11 ± 0.02	98.00 ± 0.01	97.83 ± 0.02	98.05 ± 0.04
0.0005	98.09 ± 0.01	98.01 ± 0.00	97.68 ± 0.01	98.03 ± 0.03

Table S4. Effect of finetuning the individual and averaged models for (MNIST, MLPNet): Average of the results across 5 finetuning runs as well as their standard deviation are reported for each method. Cells in orange highlight the best scores in each regime.

S3.2. For CIFAR10 + VGG11

As a recall, the original models were trained for 300 epochs at an initial learning rate of 0.05, which was decayed by a factor of 2 after every 30 epochs. The finetuning is carried out for 100 epochs at the following set of initial learning rates $\{0.01, 0.05, 0.0033, 0.0025\}$. Also, similar to training, the learning rate is decayed in the finetuning process. Note that, here finetuning at the initial learning rate of 0.01 causes model B to diverge and hence we skip the results for this setting.

For OT average, we use the weight-based alignment. Table S5 shows the best results for each method during their finetuning run.

Finetuning LR	Model A	Model B	Vanilla avg.	OT avg. (exact)
<i>Baseline Results</i>				
—	90.31	90.50	17.02	85.98
<i>Results after finetuning</i> (reported scores are at best checkpoint)				
0.01	90.29	90.53	90.39	90.73
0.005	90.36	90.47	90.16	90.64
0.0033	90.28	90.39	90.13	90.39
0.0025	90.45	90.50	89.88	90.30

Table S5. Effect of finetuning the individual and averaged models for (CIFAR10, VGG11): Model A & Model B baseline accuracies correspond to best checkpoints when originally trained for 300 epochs. Cells in orange highlight the best scores in each regime.

S3.3. For CIFAR10 + RESNET18

As a recall, the original models were trained for 300 epochs at an initial learning rate of 0.1, which was decayed by a factor of 10 at the epochs $\{150, 250\}$. The finetuning is carried out for 120 epochs at the following set of initial learning rates $\{0.1, 0.04, 0.02\}$. For OT average, we use the activation-based alignment, with mini-batch size $m = 200$.

Finetuning LR	Model A	Model B	Vanilla avg.	OT avg. (exact)
<i>Baseline Results</i>				
—	93.11	93.20	18.49	67.46
<i>Results after finetuning</i> (reported at the best checkpoint)				
(a) LR decay epochs = [20, 40, 60, 80, 100]				
0.1	93.51	93.43	93.29	93.78
0.04	93.35	93.34	93.28	93.35
0.02	93.28	93.28	93.09	92.97
(b) LR decay epochs = [40, 80]				
0.1	93.49	93.32	93.34	93.59
0.04	93.27	93.34	93.49	93.38
0.02	93.21	93.33	93.17	93.15

Table S6. Effect of finetuning the individual and averaged models for (CIFAR10, RESNET18): Model A and Model B baseline accuracies correspond to best checkpoints when originally trained for 300 epochs. Cells in orange highlight the best scores in each regime.

Table S6 shows the best results for each method during their finetuning run. The learning rate is decayed by a factor of 2 in the finetuning process as per two schedules: (a) after every 20 epochs, and (b) after every 40 epochs. These are indicated in the respective sections of the Table S6.

S4. Finetuning curves

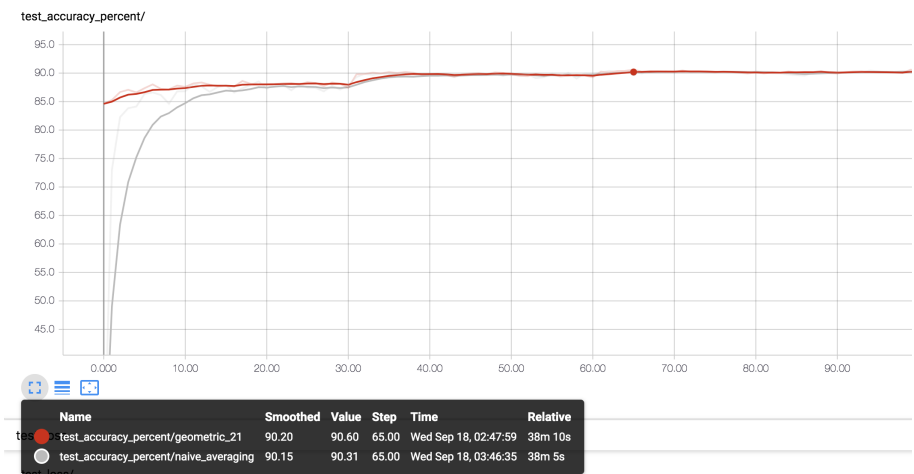


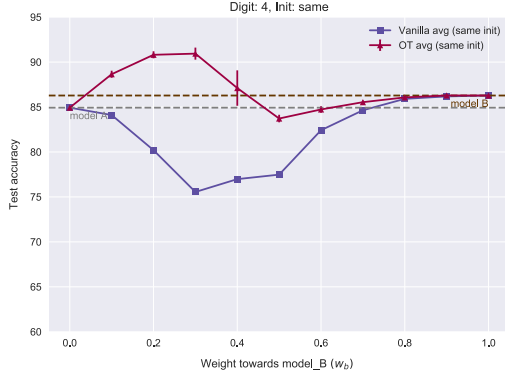
Figure S4. Illustrates the performance of OT averaging (referred to as geometric in the figure legend) and vanilla averaging during the process of retraining for CIFAR10 with VGG11.



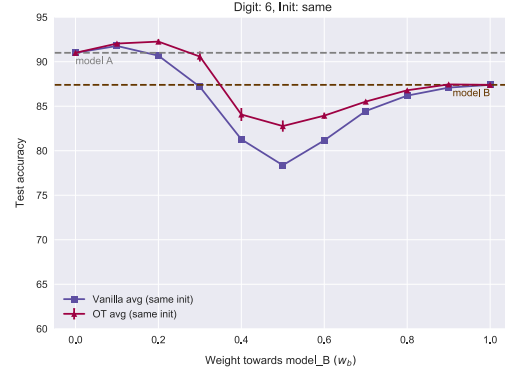
Figure S5. Retraining with reference plots of individual models. Other than that same as above.

S5. Skill Transfer: Additional Results

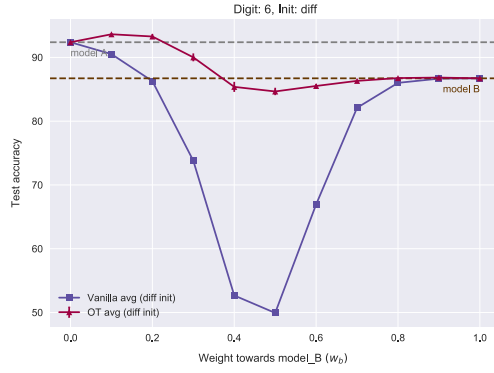
S5.1. Remaining Data Split: 10%



(a) Special digit 4, same init avg



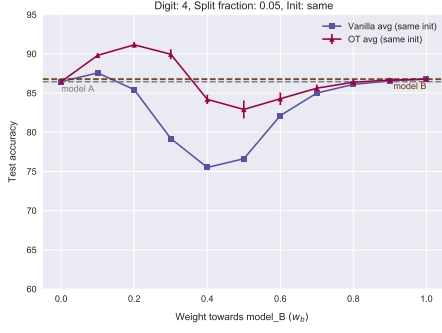
(b) Special 6, same init avg



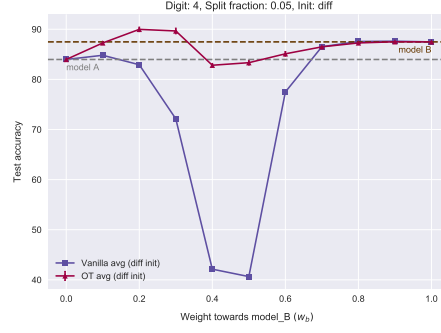
(c) Special digit 6, different init avg

Figure S6. Skill Transfer performance: Comparison results of OT based model fusion (OT avg) with vanilla averaging for different w_B . Each point for OT avg. curve (magenta colored) is obtained by activation-based alignment with a batch size $m = 400$, and we plot the mean performance over 5 seeds along with the error bars, which show the corresponding standard deviation. Here the remaining data besides the special digit, is split as 90% for model B and the other 10% for model A.

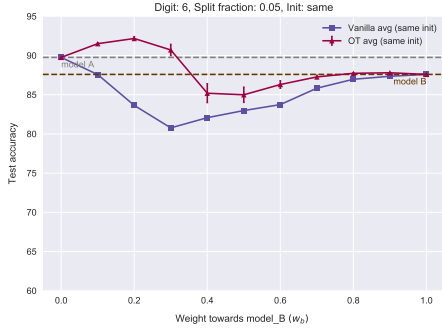
S5.2. Remaining Data Split: 5%



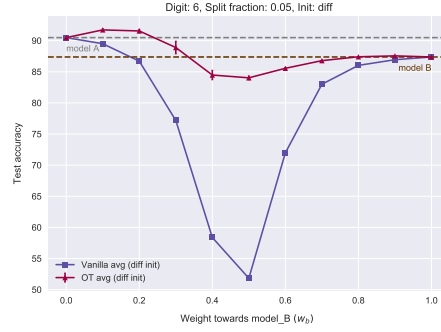
(a) Special digit 4, same init avg



(b) Special digit 4, different init avg



(c) Special digit 6, same init avg



(d) Special digit 6, different init avg

Figure S7. Skill Transfer performance: Comparison results of OT based model fusion (OT avg) with vanilla averaging for different w_B . Each point for OT avg. curve (magenta colored) is obtained by activation-based alignment with a batch size $m = 400$, and we plot the mean performance over 5 seeds along with the error bars, which show the corresponding standard deviation. Here the remaining data besides the special digit, is split as 95% for model B and the other 5% for model A.

S6. Results for one-shot skill-transfer under size constraints

Here, we present results for one-shot skill-transfer when the two models are of unequal sizes. More concretely, as an example, we consider that the hidden layers of the generalist model B are twice as wide as that of the specialist model A. Figure S8 illustrates the results for OT-based model fusion (OT average) in such a setting. Note that, here vanilla averaging can not be applied as the models are of different sizes. To the best of our knowledge, we are unaware of any other method that can allows for such one-shot skill transfer (i.e., fuse the given different size models into a single model in one-shot).

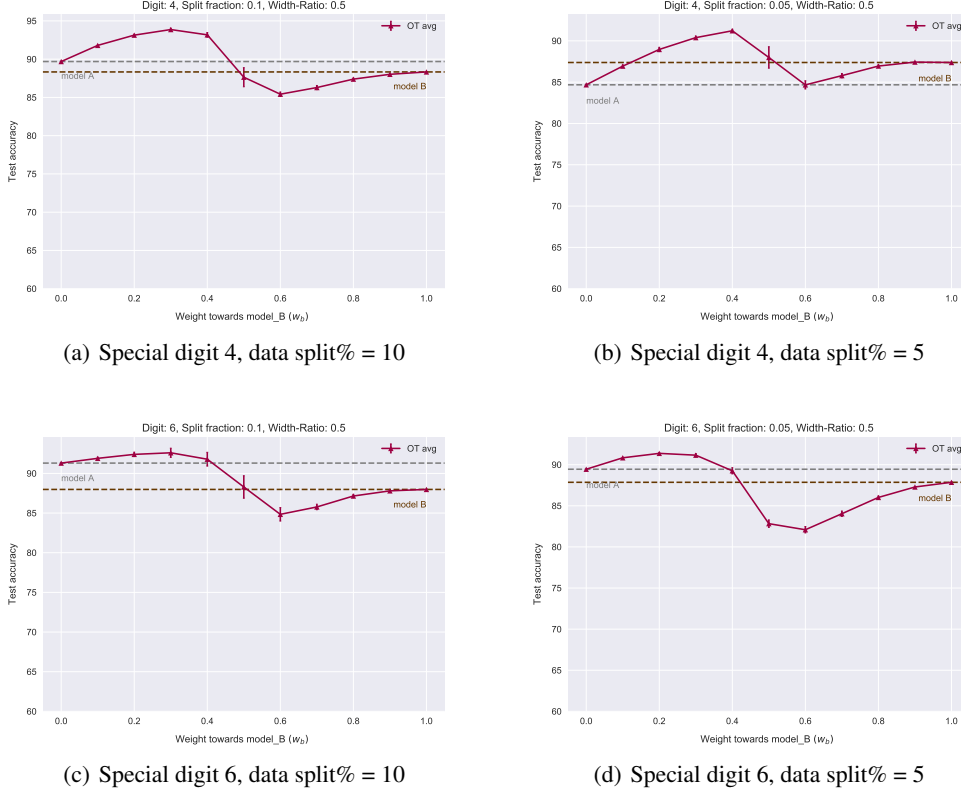


Figure S8. Skill Transfer performance for different sized models: Results of OT-based model fusion (OT avg) for different w_B . Unlike the results in the previous section, vanilla averaging is not possible here as the models are of unequal sizes. ‘Width-Ratio 0.5’ in the figure title denotes the ratio of the hidden layers sizes of model A and B. Each point for OT avg. curve (magenta colored) is obtained by activation-based alignment with a batch size $m = 400$, and we plot the mean performance over 5 seeds along with the error bars, which show the corresponding standard deviation. The data split % indicates the amount of remaining data besides the special digit which is present with model A. Model B contains 100 - data split% of this remaining data.

Rest of the technical details are identical as in the setup of Sections 6.1 in the main text and S5 in the supplementary.

S7. On the update rule in the algorithm

S7.1. Barycentric projection

The original formulation by (Monge, 1781) considers finding a mapping $f : \mathcal{S}_A \mapsto \mathcal{S}_B$, which maps points in the support of μ_A to points in the support of μ_B . However, under this formulation, the optimal transport problem is not always feasible and Kantorovich (1942) relaxed this by instead considering the optimization over the set of coupling matrices $\mathbf{T}$ (i.e., doubly stochastic matrices). Hence, a simple way to obtain the optimal map f from this coupling/transportation matrix $\mathbf{T}$ is $f(\mathcal{S}_A^i) = \mathbf{Z}_i$, where $\mathbf{Z} = \mathbf{S}_B \mathbf{T}^\top \text{diag}(\alpha^{-1})$, c.f. Ferradans et al. (2013). Essentially, this maps each point in support of A to a weighted average of the points in support of B. This is referred to as the barycentric projection.

For our algorithm in the weight-based alignment, we basically need the opposite map $f' : \mathcal{S}_B \mapsto \mathcal{S}_A$ to get the weights for the model A aligned with respect to B. This can be done by simply exchanging the above supports and transposing the matrix $\mathbf{T}$. Thus we have, $f'(\mathcal{S}_B^i) = \mathbf{Z}'_i$, where $\mathbf{Z}' = \mathbf{S}_A \mathbf{T} \text{diag}(\beta^{-1})$. In other words, we represent each point in the support of B with a weighted average of points in support of A.

Lastly, the supports in this weight-based alignment are defined by the corresponding weight matrices of the layers (Eq. (9)). Thus, implying the update in Eq. (4). Note that, when the underlying ground cost used is the squared Euclidean distance, this barycentric mapping is known to be optimal (Ambrosio et al., 2006).

S7.2. Free-support barycenters

Next, we discuss the setup and a part of the derivation of free-support barycenters proposed in (Cuturi & Doucet, 2014).

Problem formulation. Assume that the ground metric D_S is the Euclidean distance and $p = 2$. Consider the supports $\mathcal{S}_A$ and $\mathcal{S}_B$ as family of n_A and n_B points respectively in $\mathbb{R}^d$. Therefore represent them by a matrix in $\mathbb{R}^{d \times n_A}$ and $\mathbb{R}^{d \times n_B}$ respectively. If we use the notation, $\mathbf{s}_A \stackrel{\text{def}}{=} \text{diag}(\mathbf{S}_A^\top \mathbf{S}_A)$ and $\mathbf{s}_B \stackrel{\text{def}}{=} \text{diag}(\mathbf{S}_B^\top \mathbf{S}_B)$, then we can write the pairwise squared-Euclidean distances as follows:

$$\mathbf{C}_{AB} = \mathbf{s}_A \mathbf{1}_{n_B}^\top + \mathbf{1}_{n_A} \mathbf{s}_B^\top - 2 \mathbf{S}_A^\top \mathbf{S}_B \in \mathbb{R}^{n_A \times n_B}. \quad (11)$$

Now the objective in Eq. (1) of Section 3 can be written in a more compact form in the equation (12) by using the matrix inner product notation. As a recall, $\langle \mathbf{U}, \mathbf{V} \rangle = \text{tr}(\mathbf{U}^\top \mathbf{V})$, so we have:

$$\begin{aligned} \langle \mathbf{T}, \mathbf{C}_{AB} \rangle &= \langle \mathbf{T}, \mathbf{s}_A \mathbf{1}_d^\top + \mathbf{1}_d \mathbf{s}_B^\top - 2 \mathbf{S}_A^\top \mathbf{S}_B \rangle \\ &= \text{tr}(\mathbf{T}^\top \mathbf{s}_A \mathbf{1}_d^\top) + \text{tr}(\mathbf{T}^\top \mathbf{1}_d \mathbf{s}_B^\top) - 2 \langle \mathbf{T}, \mathbf{S}_A^\top \mathbf{S}_B \rangle \\ &= \mathbf{s}_A^\top \alpha + \mathbf{s}_B^\top \beta - 2 \langle \mathbf{T}, \mathbf{S}_A^\top \mathbf{S}_B \rangle. \end{aligned} \quad (12)$$

Suppose we are given the transport map $\mathbf{T}^*$ which is optimal for the above Eq (12), but we do not know the support $\mathcal{S}_B$. One way to compute it is by minimizing the above with respect to $\mathcal{S}_B$. Hence, let's discard the constant terms in $\mathbf{s}_A$ and α . Recall $\mu_A = (\alpha, \mathcal{S}_A)$ and $\mu_B = (\beta, \mathcal{S}_B)$, so we have that minimizing OT $(\mu_A, \mu_B, \mathbf{C}_{AB})$ with respect to the locations $\mathcal{S}_B$ is same as solving

$$\min_{\mathcal{S}_B \in \mathbb{R}^{d \times n_B}} \mathbf{s}_B^\top \beta - \langle \mathbf{T}^*, \mathbf{S}_A^\top \mathbf{S}_B \rangle \quad (13)$$

Quadratic Approximation. Cuturi & Doucet (2014) show that the above minimization problem in Eq. (13) is non-convex in the locations $\mathcal{S}_B$, the proof of which can be found in their work. Therefore, they resort to a local quadratic approximation mentioned in equation (14), minimizing which yields the Newton update in equation (15).

$$\mathbf{s}_B^\top \beta - \langle \mathbf{T}^*, \mathbf{S}_A^\top \mathbf{S}_B \rangle = \left\| \mathbf{S}_B \text{diag}(\beta^{1/2}) - \mathbf{S}_A \mathbf{T}^* \text{diag}(\beta^{-1/2}) \right\|^2 - \left\| \mathbf{S}_A \mathbf{T}^* \text{diag}(\beta^{-1/2}) \right\|^2 \quad (14)$$

$$\mathcal{S}_B \leftarrow \mathbf{S}_A \mathbf{T}^* \text{diag}(\beta^{-1}) \quad (15)$$

This can be interpreted as follows as follows: the matrix $T^* \text{diag}(\beta^{-1})$ has n_B columns in the simplex Σ_{n_A} and thus post-multiplying S_A with this matrix means that we are performing convex combinations of the points in S_A with weights defined by the optimal transport map T^* .

Relation to Model Fusion. Let’s come back to our algorithm where we use the weight-based alignment. Here, the locations (or the supports) are defined by the corresponding weight matrices of the layers (Eq. (9)). This bears resemblance to the update in Eq. (4) in Section 4, where the equivalent of the unknown S_B are the weights of A aligned with respect to B.

S8. Connection to the mean-field limit

Hidden layers	% Test accuracy					% Relative Gap	
	Model A	Model B	Prediction avg.	Vanilla avg.	OT avg.	Vanilla Avg.	OT avg.
[40, 20, 10]	96.69	96.91	97.50	34.82	82.91	64.07	14.44
[200, 100, 50]	98.00	97.97	98.16	47.30	93.93	51.73	4.16
[400, 200, 100]	98.13	98.09	98.21	73.51	96.70	25.09	1.45
[1000, 500, 250]	98.08	98.21	98.20	78.21	97.35	20.36	0.87
[2000, 1000, 500]	98.26	98.16	98.21	85.71	97.41	12.77	0.86

Table S7. Relative gap of OT avg. wrt the best individual model as the width of the hidden layers increases.

(Mei et al., 2019) show that for two-layer neural networks as the size of the hidden layer goes to infinity, the neurons in a layer can instead be modeled as a density which evolves with Wasserstein gradient flow. (Araújo et al., 2019) later extended this result to the multi-layer case. This in part points that given two neural networks evolving under the dynamics of Wasserstein gradient flow, combining them into one network by Wasserstein barycenter is a natural thing to consider.

We empirically show that this limit is roughly achieved in practice when the width ≈ 1000 . In particular, Table S7 illustrates that as the width of networks increases, the gap in performance of one-shot averaging (with respect to the best individual network) on MNIST decreases. As a consequence, this further implies that in the setting of finite hidden layer sizes, it would help to choose the α and β in a better way than just uniform. We aim to study this aspect in a future work.

S9. Details for the results of fusion under size constraints

Here we describe the details of the results presented in Table 4 (also presented here in Table S8 for ease). Model M_A , and model M_B , are setup so that the ratio of their hidden layer sizes is uniformly equal to ρ (referred to as the width-ratio). On MNIST + MLPNET, $\rho \in \{2, 10\}$, whereas for CIFAR10 + VGG11, $\rho \in \{2, 8\}$. This leads to the mentioned model sizes (# of parameters) for both these models. Our OT average uses activation-based alignment for both the settings described in the Table 4.

<i>Dataset + Model</i>	<i># params (M_A, M_B)</i>	M_A	M_B	<i>OT avg.</i>	<i>Finetuning</i>	
					M_B	<i>OT avg.</i>
MNIST + MLPNET	(414 K, 182 K)	98.11	97.84	95.67	98.06	98.22
	(414 K, 32 K)	98.11	97.08	96.50	97.31	97.42
CIFAR10 + VGG11	(118 M, 32 M)	91.22	90.66	86.73	90.67	90.89
	(118 M, 3 M)	91.22	89.38	88.40	89.64	89.85

Table S8. *Compressing M_A to smaller models.* The finetuning results of each method are at their best scores across different finetuning hyperparameters (like, learning rate schedules). OT avg. has the same number of parameters as M_B . Note, this is same as Table 4, and is shown here again for ease.

We illustrate with these results that a bigger model M_A can be fused into the smaller model M_B with OT average, which can be further finetuned to boost the performance. Note that vanilla averaging can not be used here since the models have different sizes (and thus different number of parameters).

As a baseline for finetuning, we also consider the smaller original model M_B . We show that even if this were to be finetuned with many different hyper-parameters, it does not outperform the performance gained by finetuning the OT average.

<i>Dataset + Model</i>	# params	Finetune			LR schedule hyper-params		
		type	LR	Epochs	Enabled	LR Decay Factor	LR Decay Epochs
MNIST + MLPNET	182 K	M_B	0.01	60	✗	—	—
	32 K		0.002				
	182 K	OT Avg.	0.01				
	32 K		0.001				
CIFAR10 + VGG11	32 M	M_B	0.01	120	✓	2.0	[20, 40, 60, 80, 100]
	3 M		0.01			1.5	[10, 30, 50, 70, 90, 110]
	32 M	OT Avg.	0.01			2.0	[20, 40, 60, 80, 100]
	3 M		0.01			2.0	[20, 40, 60, 80, 100]

Table S9. Hyper-parameters corresponding to the results for model compression presented in Tables 4. LR denotes the learning rate. For the MNIST + MLPNET setting a constant learning rate was employed, similar to its training procedure. While for the CIFAR10 + VGG11 setting, the learning rate schedule was also tuned, keeping in accordance with its training procedure as well. The # params column indicates the size of the resultant compressed (smaller) model.

For MNIST + MLPNET, we did a sweep for the following set of finetuning learning rates $\{0.01, 0.002, 0.001, 0.00067, 0.0005\}$. These correspond to scaling the training learning rate by a factor $\{1, 5, 10, 15, 20\}$ respectively. Both OT average and the model M_B were finetuned for 60 epochs using these choices as a constant learning rate.

Next, for CIFAR10 + VGG11, we additionally sweep for the hyper-parameters associated with the learning rate (LR) schedule during finetuning. Since unlike the MNIST case, the original models here used a decaying learning rate schedule. The sweep was carried out for the following set of values: LR decay factor = $\{1.2, 1.5, 2\}$, LR decay epochs = $\{[20, 40, 60, 80, 100], [10, 30, 50, 70, 90, 110], [30, 60, 90]\}$. The learning rate (LR) itself was picked from

$\{0.01, 0.005, 0.0033, 0.0025\}$ corresponding to scaling the training learning rate by a factor of $\{5, 10, 15, 20\}$ respectively, as done in Section S3.2 before.

Table S9 thus indicates the hyper-parameter choice corresponding to the best results presented in Table 4 or S8.

From sweeping on the width-ratio $\rho = 8$ for CIFAR10+ VGG11 (i.e., when the smaller model has 3M params), we found that the learning rates $\{0.01, 0.005\}$ produced the best results for both the finetuning type/methods (namely, OT average and model M_B). Thus, while sweeping on the width-ratio $\rho = 2$ for CIFAR10+ VGG11 (i.e., when the smaller model has 32M params), we reduce the hyper-parameter space by restricting the learning rate from this set $\{0.01, 0.005\}$, while still sweeping the other sets of hyper-param values, in order to save on resource costs.

Besides these best results for each method, we find that even under the same hyper-parameter configuration, finetuning OT average leads to a better performance than finetuning the smaller model M_B across multiple runs, for majority of the hyper-parameter settings. Overall, we conclude that the OT average and then finetuning successfully allows to transfer the performance from a bigger model into a smaller one.

S10. Additional results for distillation

In this section, we elaborate the results presented in relation to distilling the knowledge of a bigger model M_A into a smaller model M_B . Here, we consider that one already has a trained version of both the models and we are interested in boosting the performance of the smaller model.

We discuss two ways of approaching this problem. One is to consider the OT average of the individual models and then finetune. The other option is to use distillation (Hinton et al., 2015), where we augment the loss during finetuning with a term that essentially encourages the student’s (smaller model) logit distribution (smoothed) to be close that of the teacher’s (bigger model) logit distribution. The smoothing is done by raising the temperature (T) in the final softmax. This loss term from distillation gets is weighted by a factor of γ and a factor of $1 - \gamma$ is given to the usual finetuning loss.

The main drawback of distillation is that it requires searching for the optimal values of these hyper-parameters: temperature T and loss-weighting factor γ . As evident from the Tables S12 and S13, even for MNIST+ MLPNET, depending on the size of the smaller model, the optimal hyper-parameter values can be quite different. This can be prohibitive when dealing with larger models or datasets.

Nevertheless, we compare the performance of both these approaches in Tables S10 and S11, for the setting of MNIST+ MLPNET with width-ratio $\rho = 2, 10$ (the ratio of hidden layers sizes of the larger to the smaller model) respectively. For distillation, we consider three possible initializations for the smaller (student) model: random, model M_B , and OT average of models M_A, M_B .

M_A	M_B	Prediction avg.	OT avg.	Finetuning		Distillation		
				M_B	OT avg.	Random	M_B	OT avg.
98.11	97.84	98.10	95.49	98.04	98.19	98.18	98.22	98.30
Mean across distillation temperatures						98.13	98.17	98.26

Table S10. Compressing the bigger teacher model M_A to half its size ($\rho = 2$). The distillation scores for each student network initialization are taken for its best hyperparameter values. Both finetuning and distillation were run for 60 epochs using SGD with the same hyperparameters. Each entry has been averaged across 4 seeds. Note, this table is basically same as the Table 5, and is shown here again for ease. (We also include the results for prediction averaging here, which were skipped in the corresponding table in the main section due to space constraints.)

M_A	M_B	Prediction avg.	OT avg.	Finetuning		Distillation		
				M_B	OT avg.	Random	M_B	OT avg.
98.11	97.08	98.13	96.50	97.19	97.35	97.39	97.67	97.68
Mean across distillation temperatures						97.21	97.55	97.59

Table S11. Compressing the bigger teacher model M_A to one-tenth of its size ($\rho = 10$). The student model for distillation is initialized in 3 possible ways: random, OT avg., and model M_B . In the first row, the distillation scores are taken at its best hyperparameter values. Both finetuning and distillation were run for 60 epochs. Each entry in the table has been averaged across four seeds.

The choice of distillation hyper-parameters tried was: temperature $T = \{20, 10, 8, 4, 1\}$ and loss-weighting factor $\gamma = \{0.05, 0.1, 0.5, 0.7, 0.95, 0.99\}$. Tables S10 and S11 report the best scores across these hyper-parameter choices. Also, each each of the reported scores in the tables have been averaged across 4 seeds. The optimization parameters are same for both finetuning and distillation to ensure fair comparison. Namely, the learning rate = 0.01 and momentum = 0.5, and both were optimized with SGD for 60 epochs.

In terms of the results, we observe that initializing with OT average does the best in comparison to random and model M_B -based initializations. Further, we see that when the results for distillation with the other initializations are averaged across the distillation temperatures, the gain in performance pales in comparison to simply OT average and finetuning (c.f. Table S10).

Lastly, in Tables S12 and S13, we show the detailed results for each of the distillation initializations for each of the temperature values tried. These correspond respectively to the summarized results presented in Tables S10 and S11. We

Model Fusion via Optimal Transport

Temperature T	Distillation initializations		
	Random (γ)	M_B (γ)	OT avg. (γ)
20	98.13 (0.05)	98.20 (0.10)	98.26 (0.05)
10	98.15 (0.05)	98.19 (0.05)	98.28 (0.05)
8	98.18 (0.05)	98.22 (0.05)	98.28 (0.05)
4	98.11 (0.10)	98.21 (0.10)	98.30 (0.05)
1	98.06 (0.05)	98.04 (0.05)	98.17 (0.05)
Mean	98.13	98.17	98.26

Table S12. Distillation results for the setting of $\rho = 2$: Best results for various distillation initializations are shown for all the tried temperatures (T) values. The corresponding choice of the loss-weighting factor (γ) for these best scores is indicated next to them in brackets. Each of the scores have been averaged over four seeds. The cell in orange indicates the top result across all hyper-parameter settings and methods.

Temperature T	Distillation initializations		
	Random (γ)	M_B (γ)	OT avg. (γ)
20	97.25 (0.50)	97.61 (0.70)	97.68 (0.70)
10	97.32 (0.70)	97.67 (0.70)	97.65 (0.70)
8	97.38 (0.50)	97.67 (0.70)	97.65 (0.70)
4	97.39 (0.70)	97.53 (0.70)	97.57 (0.99)
1	96.73 (0.05)	97.28 (0.95)	97.40 (0.95)
Mean	97.21	97.55	97.59

Table S13. Distillation results for the setting of $\rho = 10$: Best results for various distillation initializations are shown for all the tried temperatures (T) values. The corresponding choice of the loss-weighting factor (γ) for these best scores is indicated next to them in brackets. Each of the scores have been averaged over four seeds. The cell in orange indicates the top result across all hyper-parameter settings and methods.

observe that distillation from OT average performs the best for most of the hyper-parameter settings, as well as in terms of the overall top performance for both the width-ratio ρ settings.

To conclude, when distillation is out of the question due to resource constraints, OT average + finetuning can go a long way. While in cases where distillation is permissible, it can be advantageous to initialize with OT average when fusing a big model into a smaller one.